

AWS 3-Tier Architecture Project Documentation

Project Overview

This project demonstrates a scalable, secure, and highly available 3-tier architecture deployed in AWS using:

- Custom VPC
- Public and Private Subnets
- Internet Gateway
- NAT Gateway
- Bastion Host
- EC2 Instances
- Application Load Balancers
- Nginx Web Tier
- Node.js App Tier
- MySQL Database Tier
- Security Groups
- Target Groups
- Auto Scaling Groups
- Launch Templates
- AMIs

Architecture Flow

```
graph TD; User --> ELB[External Application Load Balancer]; ELB --> WebTier[Web Tier (Nginx)]; WebTier --> ALB[Internal Application Load Balancer]; ALB --> AppTier[App Tier (Node.js Backend)]; AppTier --> DBTier[DB Tier (MySQL)];
```

Step 1: Create Custom VPC

Created a custom VPC to isolate the infrastructure and manage networking securely.

Example CIDR:

10.0.0.0/16

Purpose:

- Isolated cloud network
- Better security
- Controlled communication
- Custom subnetting

Step 2: Create Subnets

Created multiple subnets across two Availability Zones.

Public Subnets

Used for:

- External ALB
- Bastion Host
- Web Tier

Example:

3-tier-PubSub-A
3-tier-PubSub-B

Private Subnets

Used for:

- App Tier

Example:

3-tier-PrivateSub-A
3-tier-PrivateSub-B

Purpose:

- Secure backend communication
 - No direct internet access
-

DB Private Subnets

Used for:

- MySQL Database

Example:

```
3-tier-DBPvtSub-A
3-tier-DBPvtSub-B
```

Purpose:

- Maximum database security
 - Isolation from internet traffic
-

Step 3: Internet Gateway

Created and attached an Internet Gateway to the VPC.

Purpose:

- Enables internet access for public subnet resources

Configured public route table:

```
0.0.0.0/0 → Internet Gateway
```

Step 4: NAT Gateway

Created a NAT Gateway inside the public subnet.

Purpose:

- Allows private subnet instances to access the internet securely
- Enables package downloads and updates
- Prevents direct inbound internet access to private EC2 instances

Configured private route table:

```
0.0.0.0/0 → NAT Gateway
```

Step 5: Route Tables

Public Route Table

Associated with:

- Public subnets

Route:

0.0.0.0/0 → Internet Gateway

Private Route Table

Associated with:

- Private app subnets

Route:

0.0.0.0/0 → NAT Gateway

DB Route Table

Associated with:

- Database private subnets

Purpose:

- Database isolation
-

Step 6: Security Groups

Created separate security groups for each layer.

Frontend Security Group

Allowed:

- HTTP 80 from External ALB

- SSH from Bastion Host
-

Backend Security Group

Allowed:

- Port 4000 from Internal ALB
 - SSH from Bastion Host
-

Database Security Group

Allowed:

- MySQL 3306 from App Tier only

Purpose:

- Secure database communication
-

Step 7: Bastion Host

Created a Bastion Host in the public subnet.

Purpose:

- Secure SSH access to private EC2 instances

Connection flow:

```
graph TD
    Laptop --> BH[Bastion Host]
    BH --> EC2[Private EC2]
```

Step 8: Web Tier Setup

Launched a Web Tier EC2 instance inside the public subnet.

Installed:

- Nginx

- Frontend application

Purpose:

- Serve frontend application
- Reverse proxy API traffic

Configured nginx reverse proxy:

```
location /api/ {  
    proxy_pass http://INTERNAL-ALB-DNS/;  
}
```

Step 9: External Application Load Balancer

Created an internet-facing Application Load Balancer.

Purpose:

- Handle public traffic
- Distribute traffic across Web Tier instances
- Perform health checks
- Improve high availability

Flow:

```
Users  
↓  
External ALB  
↓  
Web Tier
```

Step 10: App Tier Setup

Launched App Tier EC2 instances inside private subnets.

Installed:

- Node.js
- PM2
- Backend application

Backend application:

- Runs on port 4000
- Provides API endpoints
- Communicates with MySQL database

Example API:

```
/health  
/transaction
```

Step 11: Internal Application Load Balancer

Created an Internal Application Load Balancer.

Purpose:

- Securely distribute API traffic
- Balance traffic across App Tier instances
- Enable backend scaling
- Improve backend high availability

Flow:

```
Web Tier  
  ↓  
Internal ALB  
  ↓  
App Tier
```

The Internal ALB is private and not internet-facing.

Step 12: Database Tier

Launched a MySQL database EC2 instance inside private DB subnets.

Purpose:

- Store application data
- Store transaction records

Database communication:

App Tier → MySQL (3306)

Only App Tier security group was allowed to access the database.

Step 13: Database Connectivity

Configured the backend Node.js application to connect to MySQL using:

```
DB_HOST
DB_USER
DB_PWD
DB_DATABASE
```

Communication used private IPs inside the VPC.

Purpose:

- Secure internal communication
 - No internet exposure
-

Step 14: Target Groups

Created Target Groups for both ALBs.

External Target Group

Used by:

External ALB → Web Tier

Port:

80

Internal Target Group

Used by:

Internal ALB → App Tier

Port:

4000

Health check endpoint:

/health

Purpose:

- Load balancing
 - Health monitoring
 - Automatic routing
 - Auto Scaling integration
-

Step 15: Health Checks

Configured health checks using:

/health

Purpose:

- Detect unhealthy instances
 - Remove unhealthy instances from traffic
 - Improve reliability
-

Step 16: AMI Creation

After validating that the application was working correctly, created AMIs of:

- Web Tier
- App Tier
- DB Tier

Purpose:

- Reusable server templates
- Faster deployment
- Consistent infrastructure

- Backup and recovery
-

Step 17: Launch Templates

Created Launch Templates for:

- Web Tier
- App Tier

Launch Templates contained:

- AMI
- Instance type
- Security groups
- Key pair

Purpose:

- Standardized EC2 launch configuration
 - Automation support
-

Step 18: Auto Scaling Groups

Created Auto Scaling Groups for:

- Web Tier
- App Tier

Configured:

- Multi-AZ deployment
- Minimum capacity
- Maximum capacity
- Desired capacity
- CPU-based scaling policy

Purpose:

- Automatic scaling
- Self-healing infrastructure
- High availability
- Fault tolerance

Auto Scaling Flow:

AMI

↓

```
Launch Template
  ↓
Auto Scaling Group
  ↓
Automatic EC2 Launch
```

Final Architecture Flow

```
Internet
  ↓
External ALB
  ↓
Web Tier (Nginx)
  ↓
Internal ALB
  ↓
App Tier (Node.js)
  ↓
DB Tier (MySQL)
```

Key Learnings

This project helped in understanding:

- AWS VPC networking
 - Public and private subnet architecture
 - Internet Gateway and NAT Gateway
 - Route tables
 - Security Groups
 - Bastion Host architecture
 - Load balancing
 - Target Groups
 - Health checks
 - Nginx reverse proxy
 - Node.js backend deployment
 - MySQL database setup
 - Internal secure communication
 - AMIs
 - Launch Templates
 - Auto Scaling Groups
 - High availability architecture
 - Scalable cloud infrastructure
-

Recreation Checklist (Complete Rebuild Guide)

This section provides the exact high-level order to recreate the entire architecture from scratch.

Complete Build Order

1. Create VPC
2. Create Public, Private, and DB Subnets
3. Create and Attach Internet Gateway
4. Create Route Tables
5. Configure Public Route → IGW
6. Create NAT Gateway
7. Configure Private Route → NAT
8. Create Security Groups
9. Launch Bastion Host
10. Launch Web Tier EC2
11. Install and Configure Nginx
12. Launch App Tier EC2
13. Install Node.js and Backend Application
14. Launch DB Tier EC2
15. Install and Configure MySQL
16. Configure DB Connectivity in Backend
17. Create External ALB + Target Group
18. Create Internal ALB + Target Group
19. Configure Health Checks
20. Configure Nginx Reverse Proxy
21. Validate Frontend and Backend APIs
22. Create AMIs
23. Create Launch Templates
24. Create Auto Scaling Groups
25. Validate Scaling and Health Checks

Important Commands Used

SSH to Bastion

```
ssh -i AWS.pem ec2-user@BASTION-PUBLIC-IP
```

SSH to Private EC2

```
ssh -i AWS.pem ec2-user@PRIVATE-IP
```

Start Backend Application

```
pm2 start index.js --name index
```

Restart Backend

```
pm2 restart index
```

Save PM2 Process

```
pm2 save
```

Verify Backend Health

```
curl http://localhost:4000/health
```

Verify Database API

```
curl http://localhost:4000/transaction
```

Example Nginx Reverse Proxy

```
location /api/ {  
    proxy_pass http://INTERNAL-ALB-DNS/;  
}
```

PM2 Status

```
pm2 status
```

MySQL Login

```
mysql -u root -p
```

Common Troubleshooting Issues

Issue: Target Group Unhealthy

Possible causes:

- Wrong health check path
- Security group issue
- Wrong target group port
- Application not running

Solution:

- Verify `/health` endpoint
 - Check port 80 or 4000
 - Verify SG rules
 - Restart backend using PM2
-

Issue: 503 Service Temporarily Unavailable

Possible causes:

- No healthy targets
- Nginx misconfiguration
- Backend not reachable

Solution:

- Check ALB target group health
 - Verify Nginx proxy_pass
 - Check backend health endpoint
-

Issue: Database Connection Failure

Possible causes:

- DB private IP changed
- Wrong DB credentials
- Port 3306 blocked

Solution:

- Update `DbConfig.js`
 - Restart PM2
 - Verify DB SG rules
-

Issue: Cannot SSH to Private EC2

Solution:

- SSH through Bastion Host
 - Verify key pair
 - Verify SSH SG rules
-

Conclusion

This project successfully implemented a scalable, highly available, and secure 3-tier AWS architecture using AWS networking, compute, load balancing, and scaling services following production-style cloud architecture best practices.